

Hyperparameter Tuning Strategies

Spend a fixed search budget where it teaches you the most

Optimization in Practice · Foundations of Machine Learning



Search Spaces Grow Combinatorially

Interactions — The best learning rate can depend on batch size and regularization.

Expensive trials — Each candidate may require a full cross-validated fit.

Grid explosion — A few values across many dimensions become thousands of combinations.

$$N_{\text{configurations}} = v^h$$

Grid Search Is the Exhaustive Baseline

Strength — Find the best point inside the stated grid.

Weakness — Spend equal compute on promising and useless regions.

Hidden cost — Configurations multiply again by the number of CV folds.

	Column A	Column B
Row A	✓	✗
Row B	✗	✓

Random Search Covers Important Axes Better

Grid — Repeats a small, fixed set of values along every axis.

Random — Samples a fresh value per axis on every trial, for the same budget.

Why it wins — Often only a few hyperparameters strongly control performance.

Grid: $3 \times 3 = 9$ trials, only 3 distinct x-values Random: 9 trials, 9 distinct x-values



unimportant axis (vertical)



unimportant axis (vertical)

If only the horizontal axis affects validation score:

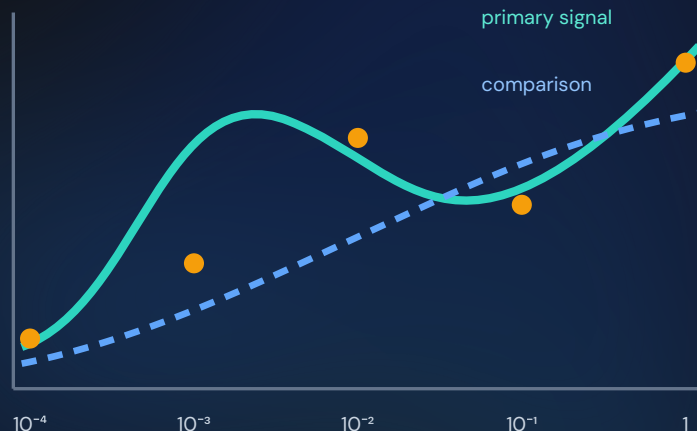
grid search effectively tested only 3 useful settings,
random search effectively tested 9 — same trial budget.

Sample Multiplicative Parameters on a Log Scale

Orders of magnitude — 0.001 → 0.01 matters like 0.01 → 0.1.

Linear sampling — Overrepresents the high end of a wide interval.

Log-uniform — Gives each power-of-ten band equal attention.



$$\log_{10} h \sim \mathcal{U}(-4, 0)$$

RandomizedSearchCV

Fixed budget

`n_iter` controls the number of configurations.

Distributions

Describe ranges instead of enumerating every point.

```
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV

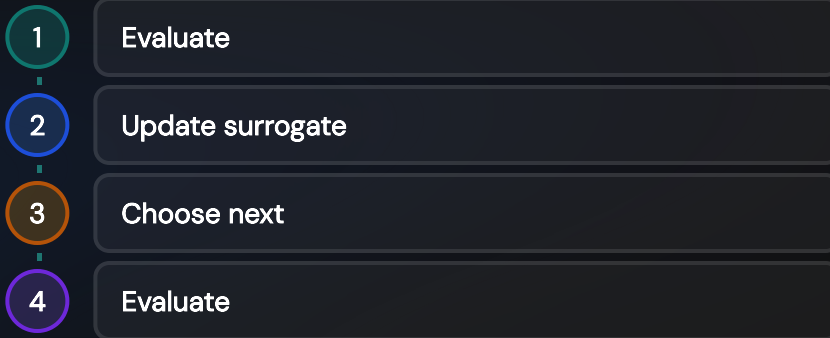
search = RandomizedSearchCV(
    SVC(),
    {"C": loguniform(1e-2, 1e2),
     "gamma": loguniform(1e-4, 1)},
    n_iter=20, cv=5, random_state=0,
)
search.fit(X, y)
```

Bayesian Optimization Learns From Trials

Surrogate model — Estimate performance and uncertainty across the search space.

Acquisition — Choose the next candidate where promise or uncertainty is high.

Tradeoff — Fewer expensive trials at the cost of a more complex search algorithm.



A Practical Broad-to-Narrow Workflow

Start broad — Random search over wide, sensible ranges.

Narrow — Refine around promising regions.

Lock decisions — Select the final configuration with validation or CV.

Test once — Use the untouched test set only at the end.

1

Wide random search

2

Focused search

3

Final model

4

One test

Tune Systematically

Budget

Count evaluations.

Scale

Use log ranges when appropriate.

Discipline

Never tune on test.

Next: broader context—fairness, interpretability, and responsible deployment.